

The json Module in Python

Reading, writing, and troubleshooting JSON data

JSON (JavaScript Object Notation) is the most common format for exchanging structured data over APIs, config files, and NoSQL stores. Python's built-in `json` module converts between JSON text and native Python objects — no installation required. This guide covers the four core functions, type mapping, formatting options, nested data, custom objects, error handling, and common pitfalls.

Function	Direction	Use with
<code>json.load(f)</code>	JSON file → Python object	an open file handle
<code>json.dump(obj, f)</code>	Python object → JSON file	an open file handle
<code>json.loads(s)</code>	JSON string → Python object	a string already in memory
<code>json.dumps(obj)</code>	Python object → JSON string	returns a string

■ Remembering which is which

The extra "s" in `dumps` / `loads` stands for **string**. No "s" means it works directly with a file.

1. Reading JSON

1.1 From a file — `json.load()`

```
import json

with open("data.json", encoding="utf-8") as f:
    data = json.load(f) # parses the file directly into a Python object

print(data["name"])
print(data["tags"][0])
```

1.2 From a string — `json.loads()`

Use this when the JSON is already in memory, such as text from an API response or a message queue:

```
import json

text = '{"name": "Amina", "age": 29, "active": true}'
data = json.loads(text)
print(data["name"], data["age"])
```

With the `requests` library, API responses often have a built-in `.json()` helper that does this for you:

```
import requests

response = requests.get("https://api.example.com/users/1")
data = response.json() # equivalent to json.loads(response.text)
```

2. Writing JSON

2.1 To a file — `json.dump()`

```
import json

data = {
    "name": "Amina",
    "age": 29,
    "skills": ["python", "sql", "docker"],
    "active": True,
}

with open("output.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2, ensure_ascii=False)
```

2.2 To a string — `json.dumps()`

```
text = json.dumps(data, indent=2)
print(text)
```

2.3 Useful keyword arguments

Argument	Effect
<code>indent=2</code>	Pretty-print with 2-space indentation (omit for compact single-line output)
<code>ensure_ascii=False</code>	Keep accented / non-Latin characters readable instead of escaping to <code>\uXXXX</code>
<code>sort_keys=True</code>	Sort dictionary keys alphabetically in the output
<code>separators=(",", ":")</code>	Remove extra whitespace — useful for the smallest possible output
<code>default=func</code>	Custom function to serialize objects json doesn't know how to handle

■ `ensure_ascii=False`

Without this, non-ASCII characters like accented letters, Arabic script, or emoji get escaped into `\uXXXX` sequences. Setting it to `False` keeps the output human-readable while remaining valid JSON.

3. Type Mapping Between JSON and Python

JSON	Python (loads)	Python → JSON (dumps)
object	dict	dict → object
array	list	list, tuple → array
string	str	str → string
number (int)	int	int → number
number (float)	float	float → number
true / false	True / False	True/False → true/false
null	None	None → null

Note that JSON has no tuple or set type — both get converted to a JSON array, and a Python set isn't serializable at all without custom handling since sets are unordered.

4. Working with Nested Data

4.1 Safe access with .get()

Real-world JSON is often deeply nested, and keys aren't always guaranteed to exist. Chained `.get()` calls with defaults avoid `KeyError`:

```
data = {
    "user": {"name": "Karim", "address": {"city": "Cairo"}},
    "orders": [
        {"id": 1, "total": 42.5},
        {"id": 2, "total": 17.0},
    ],
}

city = data.get("user", {}).get("address", {}).get("city", "Unknown")
order_total = sum(order["total"] for order in data["orders"])

print(city, order_total)
```

4.2 Flattening nested JSON with pandas

For deeply nested JSON that needs to become a flat table (e.g. for CSV export or analysis), `pandas.json_normalize` is the standard tool:

```
import json
import pandas as pd

with open("nested.json") as f:
    raw = json.load(f)

df = pd.json_normalize(raw, record_path="orders", meta=["user_id"])
```

5. Serializing Custom Objects and Dates

JSON has no native date, datetime, or custom-class type. Trying to serialize one directly raises `TypeError: Object of type datetime is not JSON serializable`.

5.1 Using the default argument

```
import json
from datetime import datetime

def default_serializer(obj):
    if isinstance(obj, datetime):
        return obj.isoformat()
    raise TypeError(f"Type {type(obj)} not serializable")

data = {"name": "Amina", "created": datetime.now()}
text = json.dumps(data, default=default_serializer)
```

5.2 Serializing a custom class

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

def user_to_dict(obj):
    if isinstance(obj, User):
        return {"name": obj.name, "age": obj.age}
    raise TypeError(f"Type {type(obj)} not serializable")

user = User("Karim", 31)
text = json.dumps(user, default=user_to_dict)
```

5.3 Deserializing back into an object with object_hook

```
def dict_to_user(d):
    if "name" in d and "age" in d:
        return User(d["name"], d["age"])
    return d

restored = json.loads(text, object_hook=dict_to_user)
print(restored.name, restored.age)
```

6. Error Handling

Malformed JSON raises `json.JSONDecodeError` (a subclass of `ValueError`). Always catch it when parsing untrusted or unpredictable input, such as API responses or user uploads:

```
import json

text = '{"name": "Amina", "age": }' # malformed - missing value

try:
    data = json.loads(text)
except json.JSONDecodeError as e:
    print(f"Invalid JSON: {e.msg} at line {e.lineno}, column {e.colno}")
```

■ JSON syntax is stricter than Python literals

JSON requires **double quotes** for strings and keys, and does **not** allow trailing commas before a closing brace or bracket. A Python dict literal with single quotes, or a JSON-like string copy-pasted with a trailing comma, will fail to parse.

7. Common Pitfalls

- **Forgetting `encoding="utf-8"`:** always specify it when opening files, especially with non-English text, to avoid platform-dependent default encodings.
- **Assuming keys always exist:** use `.get(key, default)` instead of direct indexing on data from external sources.
- **Serializing unsupported types directly:** datetimes, sets, and custom classes need a `default` function or manual conversion first.
- **Trailing commas:** unlike Python dict/list literals, JSON does not allow a trailing comma before `}` or `]` — it's a hard parse error.
- **Single vs. double quotes:** JSON strings must use double quotes; a Python-style dict with single quotes is not valid JSON text.
- **Very large files:** `json.load()` reads the entire file into memory. For huge JSON files that don't fit in memory, use a streaming parser like `ijson` instead.
- **Integer precision:** JSON numbers map to Python `int/float`, but very large integers from JavaScript sources can lose precision if the origin used 64-bit floats — worth verifying when exact large IDs matter.

Task	Best approach
Parse a JSON file	<code>json.load(f)</code>
Parse a JSON string / API response	<code>json.loads(s)</code> or <code>response.json()</code>
Write pretty-printed JSON	<code>json.dump(obj, f, indent=2, ensure_ascii=False)</code>
Serialize datetime or custom objects	<code>json.dumps(obj, default=your_function)</code>
Handle malformed input safely	<code>try/except json.JSONDecodeError</code>
Flatten deeply nested JSON to a table	<code>pandas.json_normalize</code>